CS 301
High-Performance Computing

Lab 02 - Matrix Multiplication
Performance Analysis

Khushi Pandya (202301406)
Aalok Thakkar (202301429)

February 18, 2026

Contents

1 Introduction

Matrix multiplication is a fundamental operation in scientific computing and high-performance applications. The primary objective of this lab is to evaluate the performance of different matrix multiplication algorithms on two computing platforms: the Lab PC and the HPC Cluster.

This lab focuses on three specific methods:

- **Problem A-1:** Conventional matrix multiplication, evaluating all six possible loop orderings.
- **Problem B-1:** Matrix multiplication using the transpose method to improve data locality.
- **Problem C-1:** Block (or tiled) matrix multiplication using a divide-and-conquer approach to maximize cache utilization.

We measure and compare **runtime**, **throughput**, and **MFLOPS** across matrix sizes from 2^2 to 2^{12}, using both 4-byte integers and 8-byte floating-point values. This analysis will highlight the impact of memory hierarchy, algorithmic optimization, and hardware capabilities on performance.

2 Hardware Details

2.1 Hardware Details for LAB207 PCs

Table 1: Hardware Specifications of Lab 207 PCs

Specification	**Details**
Architecture	x86_64
CPU(s)	12
Threads per core	2
Cores per socket	6
Socket(s)	1
CPU Model	12th Gen Intel(R) Core(TM) i5-12500
CPU Max MHz	4600
CPU Min MHz	800
L1d Cache	288 KB
L1i Cache	192 KB
L2 Cache	7.5 MB
L3 Cache	18 MB

2.2 Hardware Details for HPC Cluster (Node gics1)

Table 2: Hardware Specifications of HPC Cluster Node gics1

Specification	**Details**
Architecture	x86_64
CPU(s)	16
Threads per core	1
Cores per socket	8
Socket(s)	2
CPU Model	Intel(R) Xeon(R) CPU E5-2640 v3 @ 2.60GHz
CPU MHz	1288.73
L1d Cache	32 KB
L1i Cache	32 KB
L2 Cache	256 KB
L3 Cache	20 MB

3 Problem Description

This lab implements three matrix multiplication approaches to investigate computational and memory performance:

1. **Conventional Matrix Multiplication (Problem A-1):** Computes the product using nested loops in all possible orderings. This highlights the effect of loop order on cache performance and runtime.
2. **Matrix Multiplication using Transpose (Problem B-1):** Improves spatial locality by transposing one of the matrices, thereby reducing cache misses and improving throughput.
3. **Block Matrix Multiplication (Problem C-1):** Divides matrices into smaller blocks and multiplies them using a divide-and-conquer strategy. This method exploits temporal locality to maximize cache utilization.

The goals of this lab are to:

- Analyze the effect of matrix size on **runtime** and **MFLOPS**.
- Compare memory-bound versus compute-bound behavior of each algorithm.
- Examine the performance improvement achieved by algorithmic optimizations.

4 Benchmarking Methodology

All algorithms were executed on both the Lab PC and the HPC Cluster. The methodology includes:

- Measuring **execution time** for various matrix sizes.
- Computing **throughput** using:

$$\text{Throughput} = \frac{\text{sizeof(element)} \times N^2 \times \text{operations}}{\text{Runtime (s)}}$$

- Calculating **MFLOPS** using:

$$\text{MFLOPS} = \frac{2N^3}{\text{Runtime (s)} \times 10^6}$$

All experiments were repeated five times to capture runtime fluctuations and support statistical analysis. The results are analyzed in terms of memory-bound vs compute-bound characteristics, and the effects of cache optimizations are highlighted.

5 Graphical Results

5.1 Problem A-1: Conventional Matrix Multiplication

5.1.1 Throughput using Lab PC - ijk

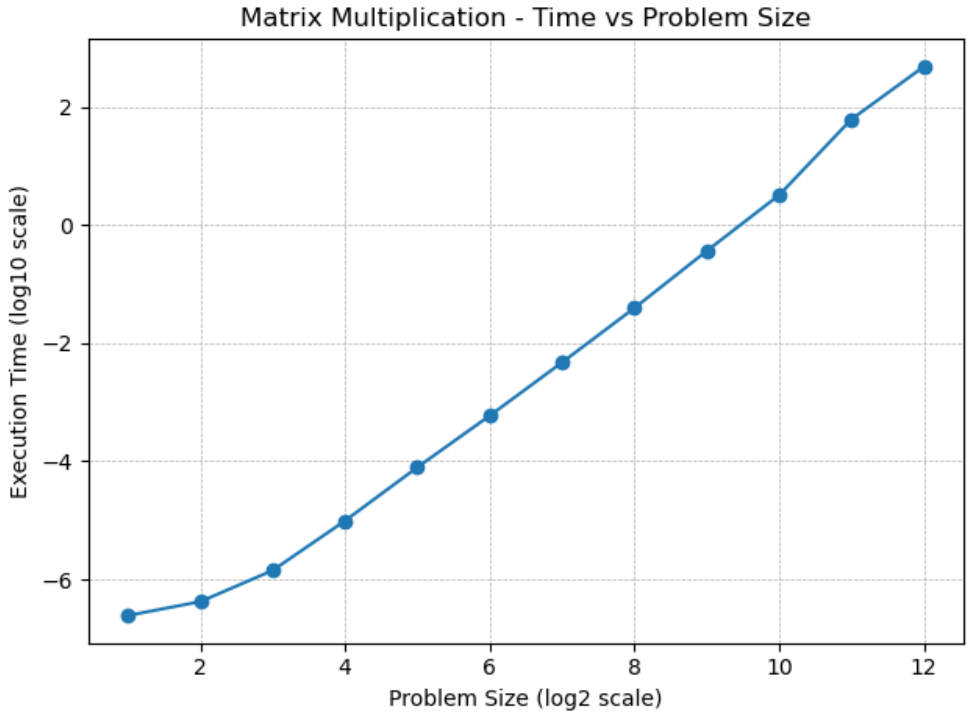

Figure 1: Throughput vs Problem Size for Conventional Multiplication on Lab PC

5.1.2 Throughput using HPC Cluster -ijk

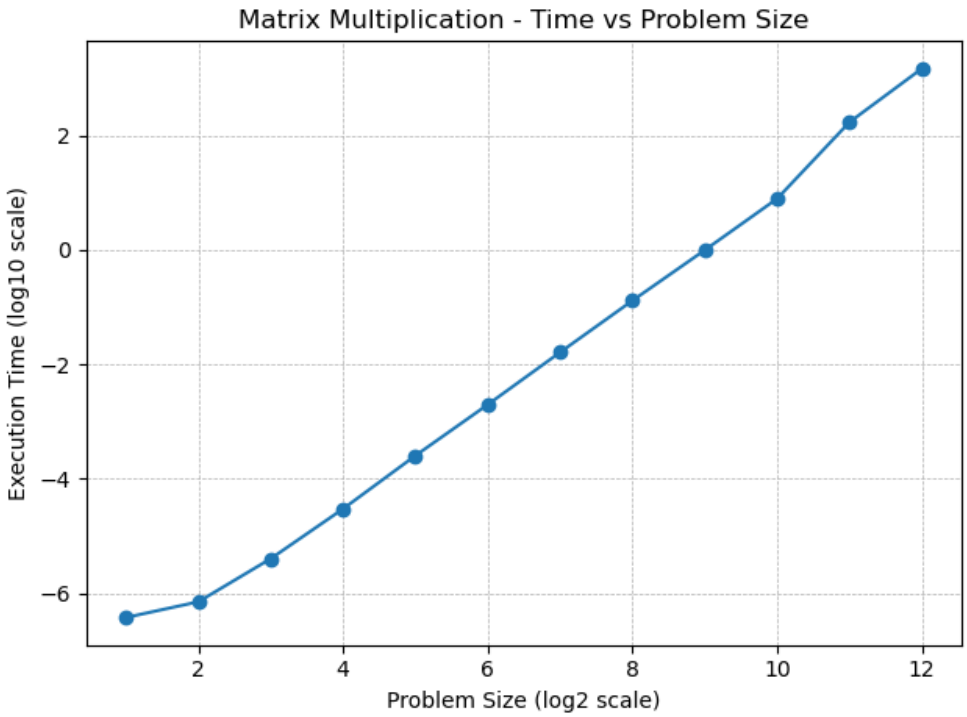

Figure 2: Throughput vs Problem Size for Conventional Multiplication on HPC Cluster

5.1.3 Throughput using Lab PC - ikj

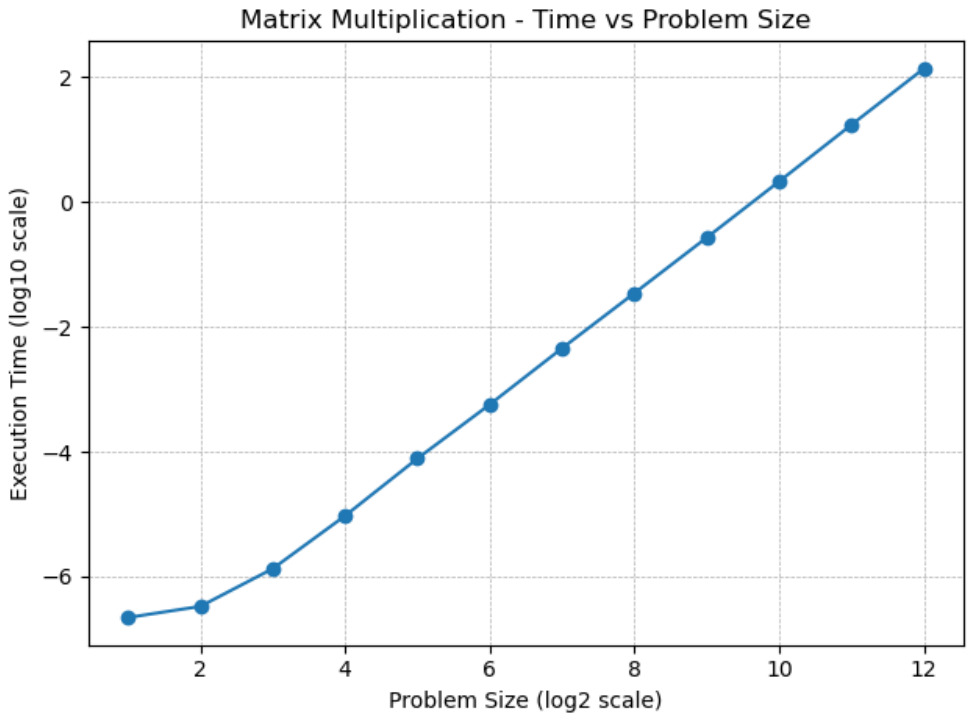

Figure 3: Throughput vs Problem Size for Conventional Multiplication on Lab PC

5.1.4 Throughput using HPC Cluster -ikj

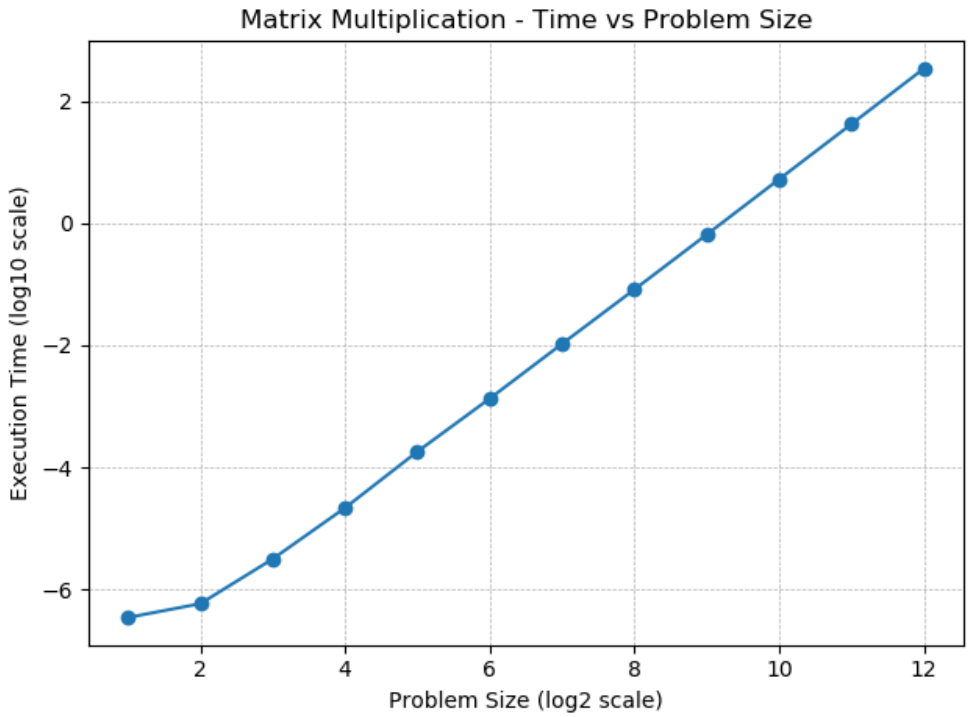

Figure 4: Throughput vs Problem Size for Conventional Multiplication on HPC Cluster

5.1.5 Throughput using Lab PC - jik

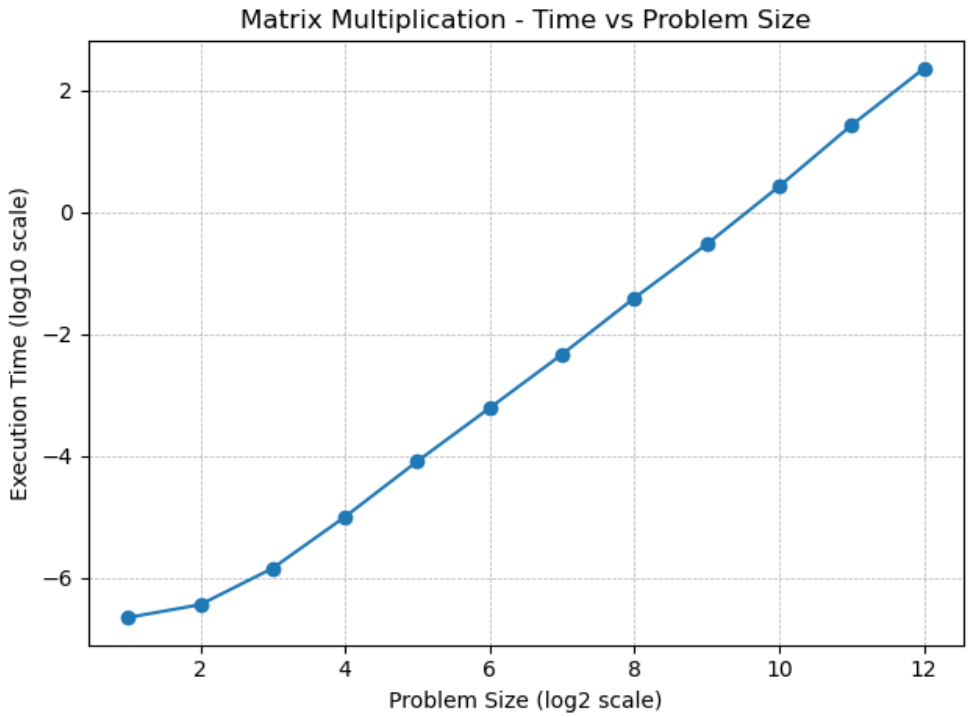

Figure 5: Throughput vs Problem Size for Conventional Multiplication on Lab PC

5.1.6 Throughput using HPC Cluster -jik

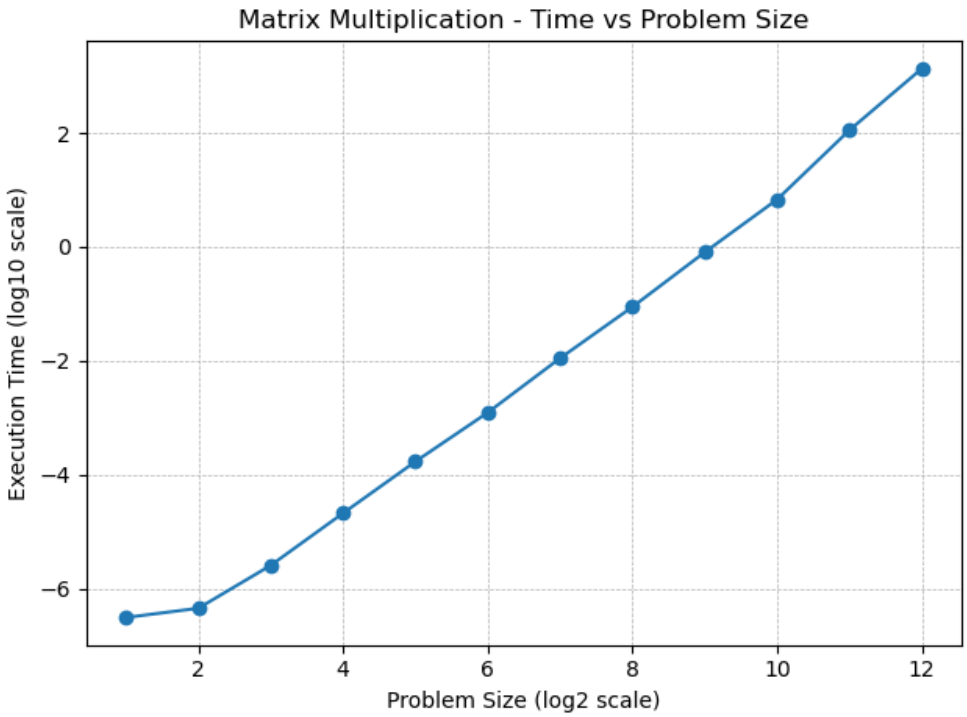

Figure 6: Throughput vs Problem Size for Conventional Multiplication on HPC Cluster

5.1.7 Throughput using Lab PC - jki

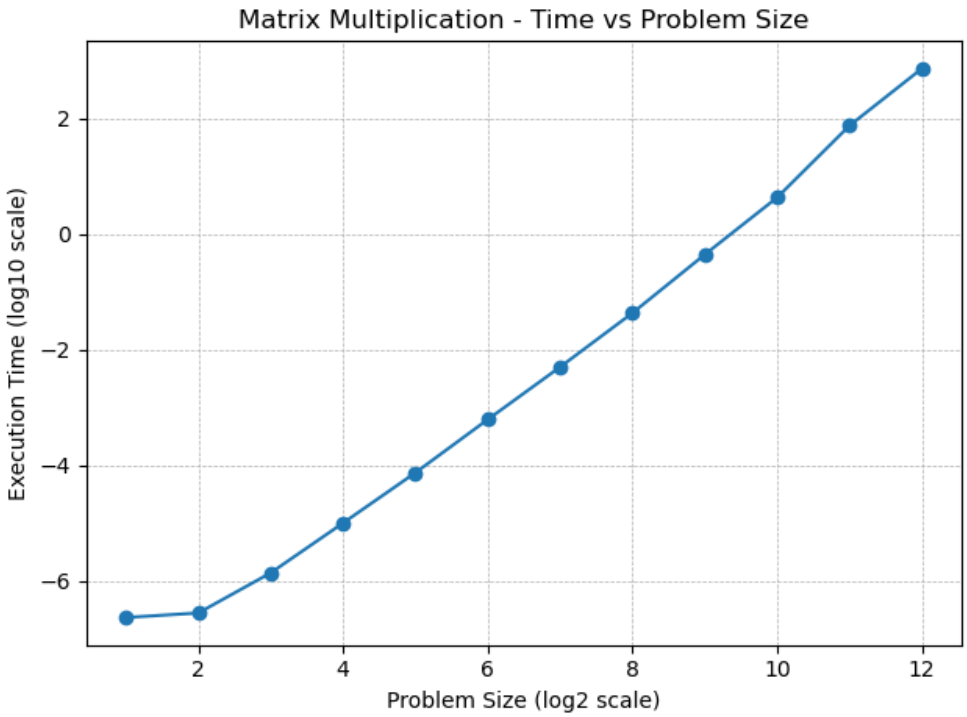

Figure 7: Throughput vs Problem Size for Conventional Multiplication on Lab PC

5.1.8 Throughput using HPC Cluster -jki

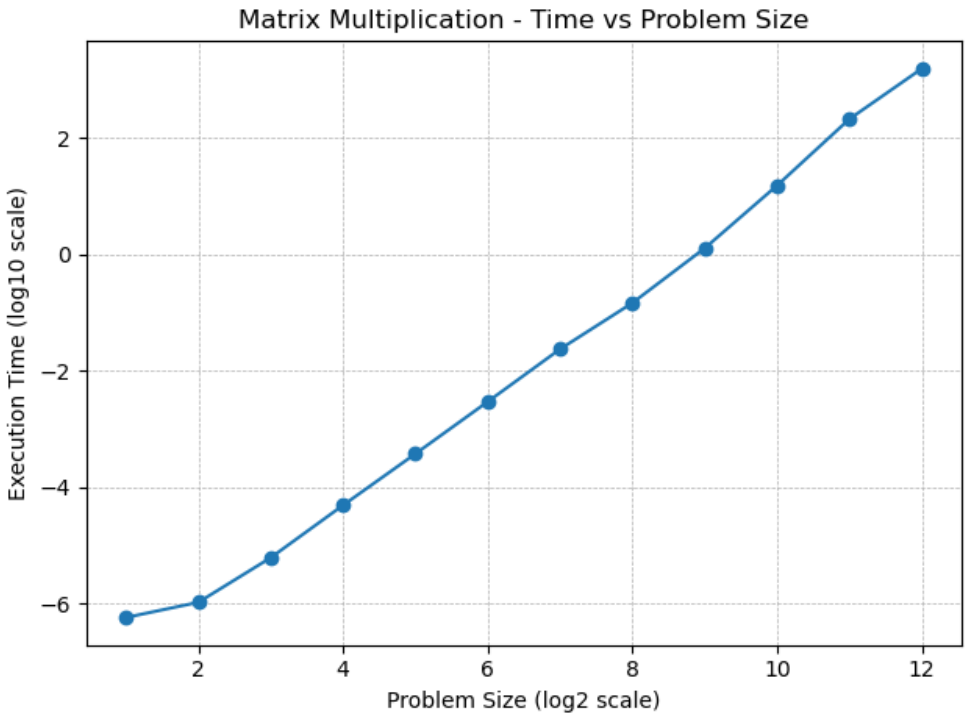

Figure 8: Throughput vs Problem Size for Conventional Multiplication on HPC Cluster

5.1.9 Throughput using Lab PC - kij

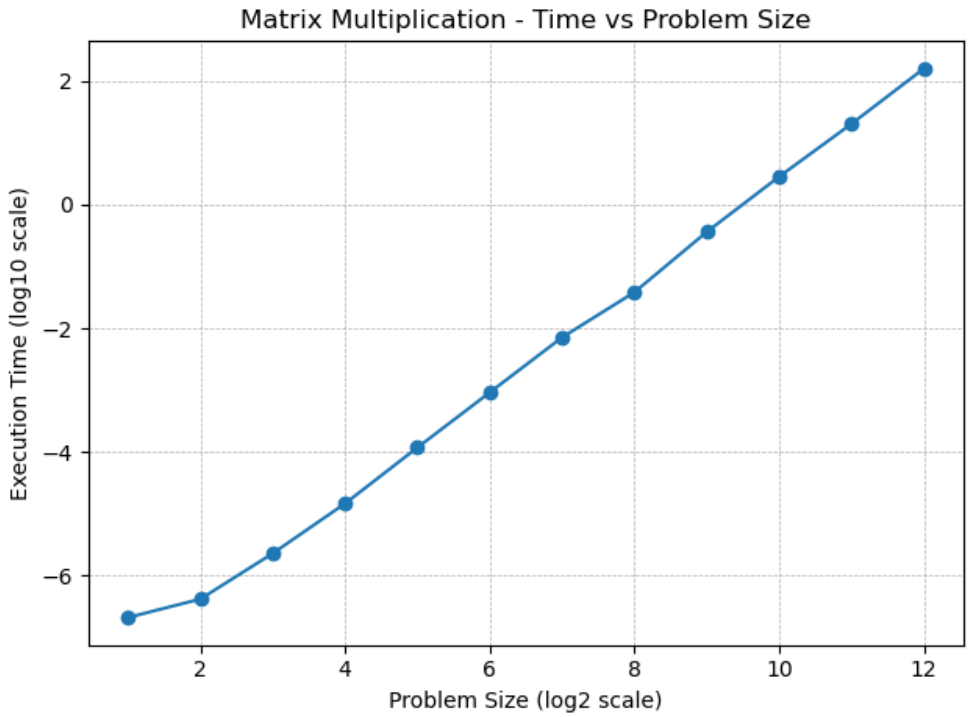

Figure 9: Throughput vs Problem Size for Conventional Multiplication on Lab PC

5.1.10 Throughput using HPC Cluster -kij

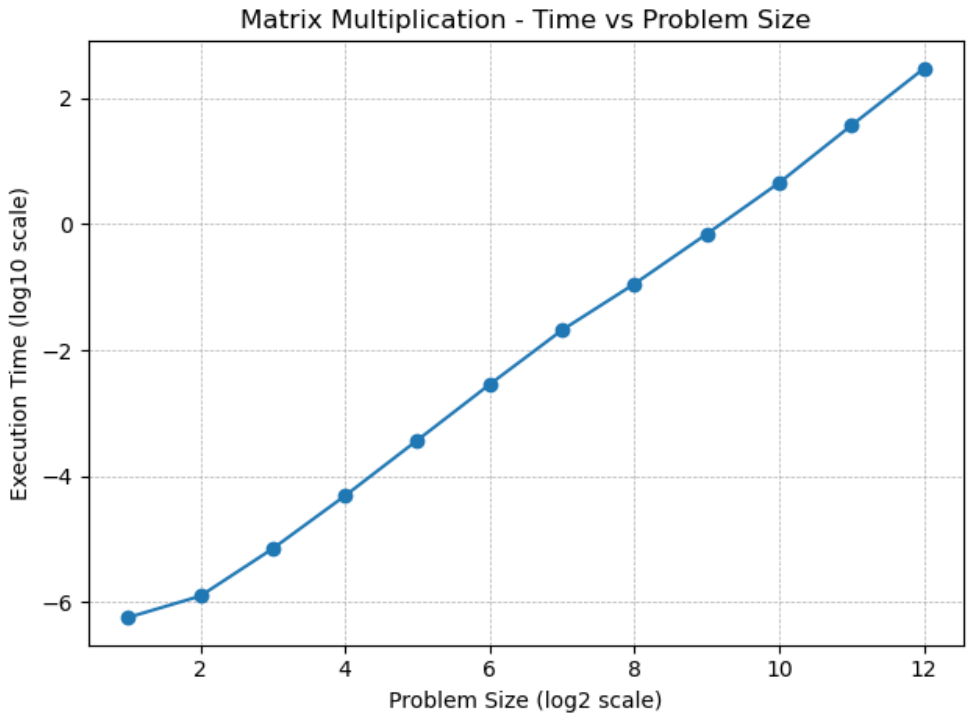

Figure 10: Throughput vs Problem Size for Conventional Multiplication on HPC Cluster

5.1.11 Throughput using Lab PC - kji

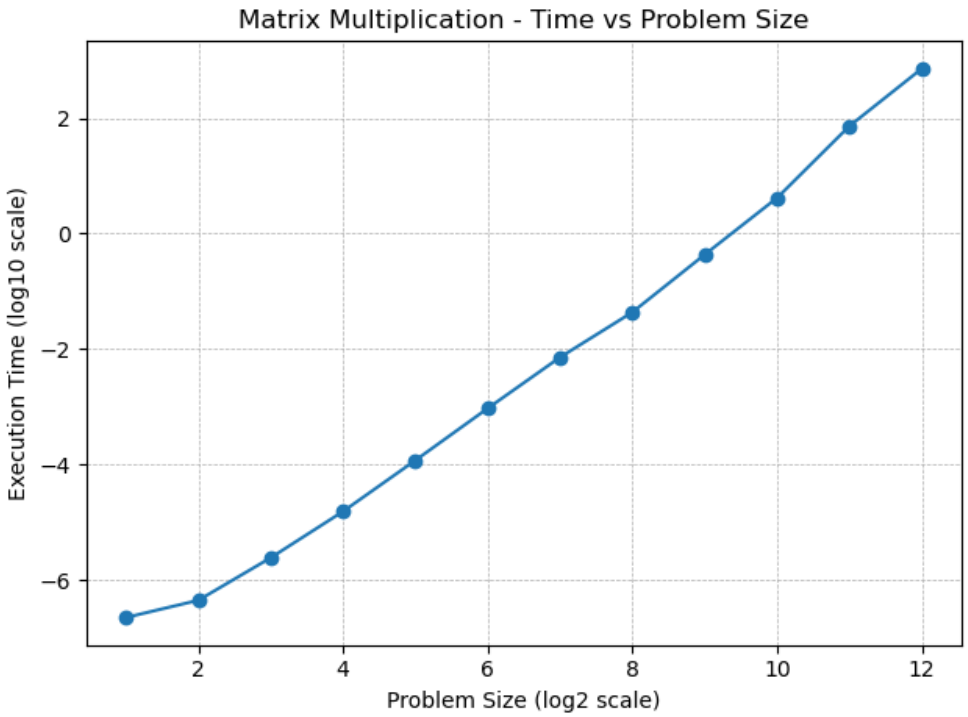

Figure 11: Throughput vs Problem Size for Conventional Multiplication on Lab PC

5.1.12 Throughput using HPC Cluster -kji

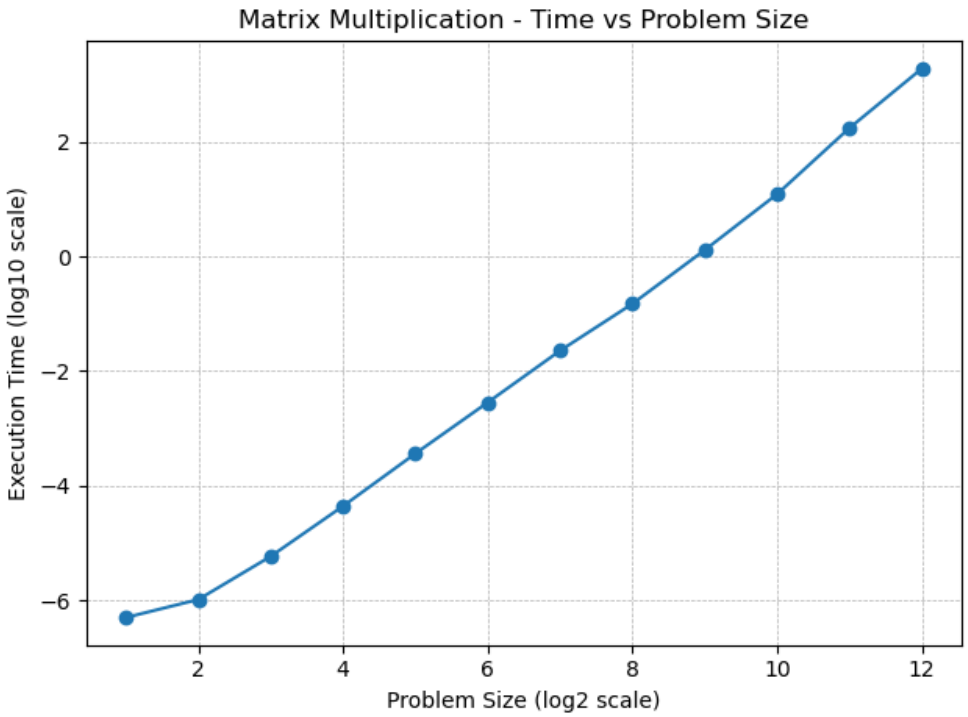

Figure 12: Throughput vs Problem Size for Conventional Multiplication on HPC Cluster

5.2 Problem B-1: Matrix Multiplication using Transpose

5.2.1 Throughput using Lab PC

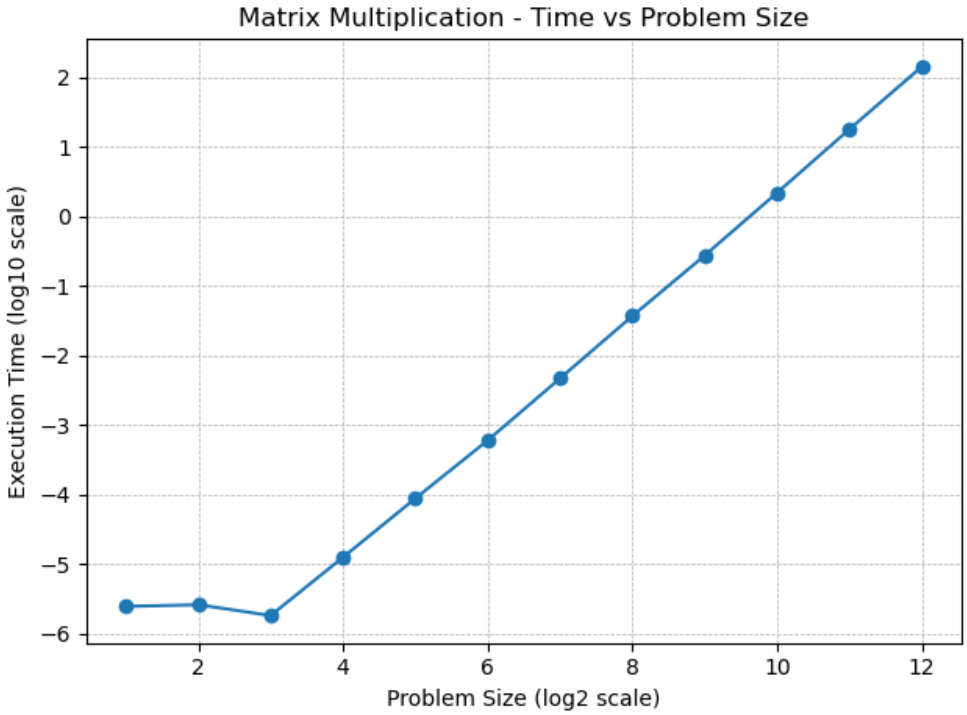

Figure 13: Throughput vs Problem Size for Transpose Multiplication on Lab PC

5.2.2 Throughput using HPC Cluster

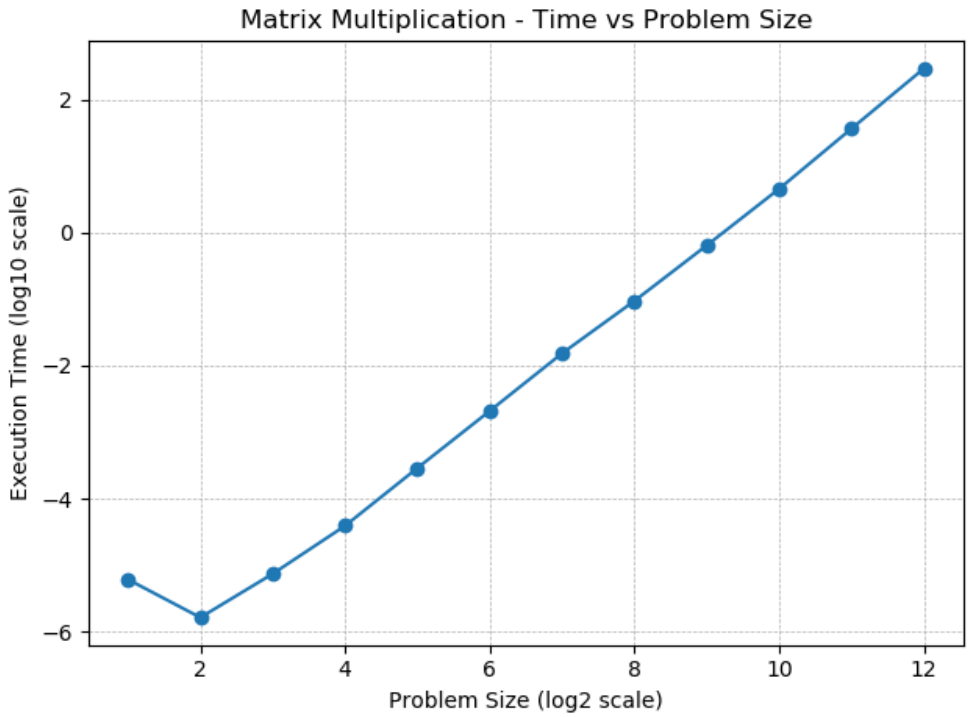

Figure 14: Throughput vs Problem Size for Transpose Multiplication on HPC Cluster

5.3 Problem C-1: Block Matrix Multiplication

5.3.1 Throughput using Lab PC

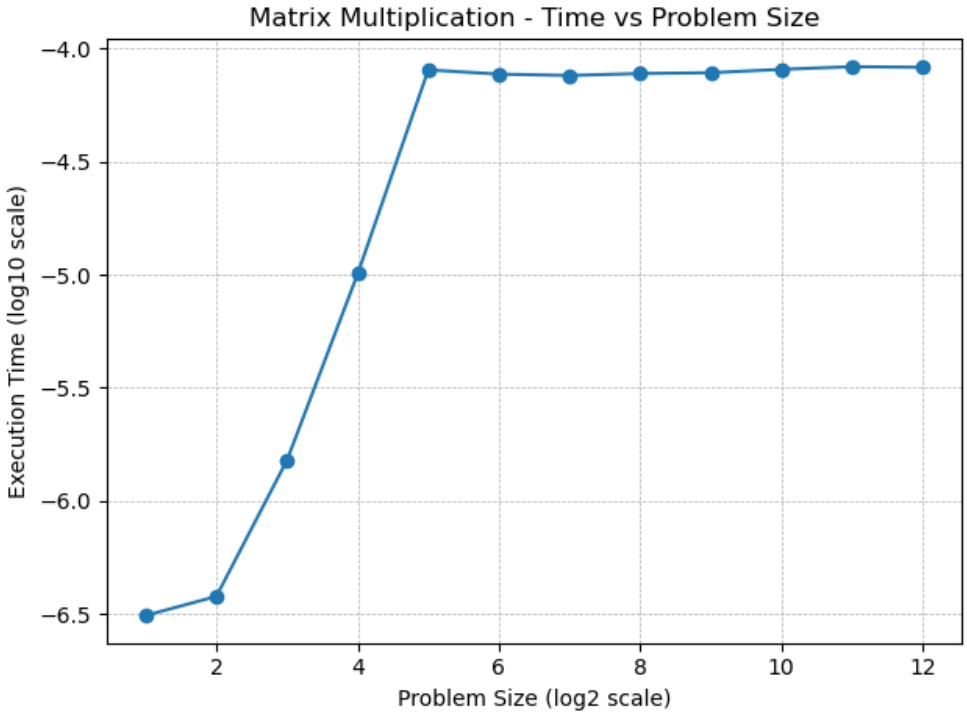

Figure 15: Throughput vs Problem Size for Block Multiplication on Lab PC

5.3.2 Throughput using HPC Cluster

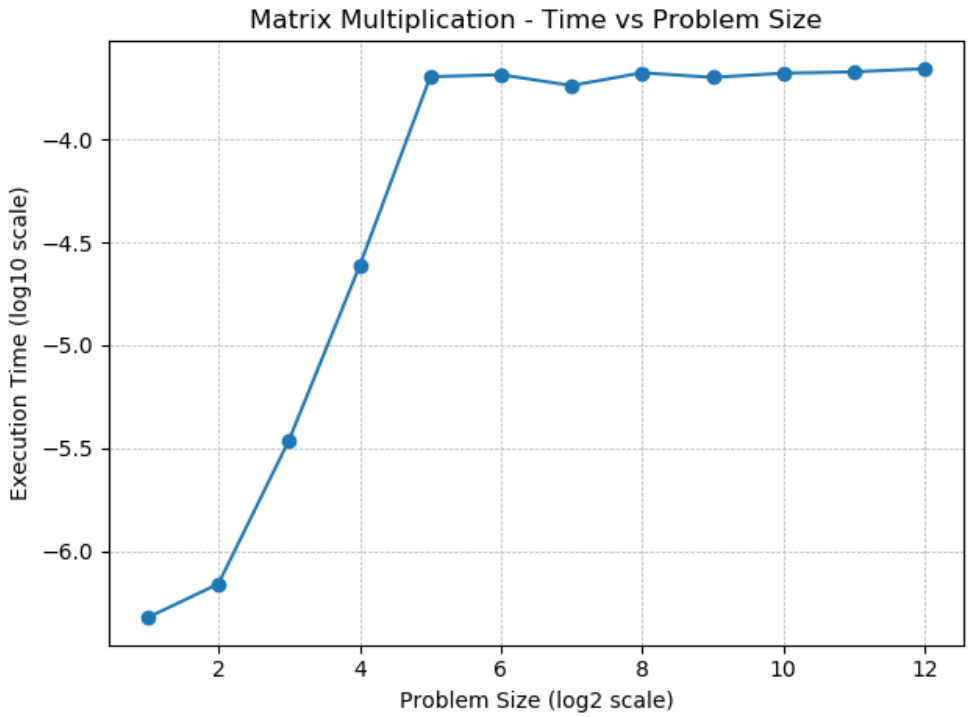

Figure 16: Throughput vs Problem Size for Block Multiplication on HPC Cluster

6 Conclusion

- Execution time grows rapidly with increasing matrix size, consistent with the $O(N^3)$ computational complexity of matrix multiplication.
- The conventional implementation experiences reduced throughput for larger matrices, primarily due to increased cache misses.
- Using the transpose method enhances spatial locality, leading to improved throughput compared to the naive approach.
- The block (tiled) multiplication strategy delivers the best performance by effectively reusing cache and minimizing memory access overhead.
- The HPC Cluster demonstrates superior performance relative to the Lab PC, owing to its higher core count and larger cache capacity.
- The measured throughput remains below the theoretical peak performance, mainly because of memory bandwidth constraints.